

```
# =====  
#           STEP 5: CHURN ANALYSIS  
# =====  
  
# =====  
# STEP 1: IMPORT REQUIRED LIBRARIES  
# =====  
  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
  
print("Libraries imported successfully.")  
  
# =====  
# STEP 2: LOAD CUSTOMER RFM SEGMENTATION DATA  
# =====  
  
# This file was created during Phase 3  
rfm = pd.read_csv("customer_rfm_segments.csv")  
  
print("RFM dataset loaded successfully.")  
  
print("\nDataset Shape:")  
print(rfm.shape)
```

```
print("\nFirst 5 Records:")
display(rfm.head())
```

```
# =====
# STEP 3: CHECK COLUMN NAMES
# =====
```

```
print("\nAvailable Columns:")
print(rfm.columns.tolist())
```

```
# =====
# STEP 4: STANDARDIZE COLUMN NAMES
# =====
```

```
# Remove unwanted spaces from column names
rfm.columns = (
    rfm.columns
    .str.strip()
)
```

```
print("\nStandardized Columns:")
print(rfm.columns.tolist())
```

```
# =====  
# STEP 5: CHECK REQUIRED COLUMNS  
# =====  
  
required_columns = [  
    "customer_id",  
    "recency",  
    "frequency",  
    "monetary",  
    "R_Score",  
    "F_Score",  
    "M_Score",  
    "RFM_Score",  
    "churn"  
]  
  
missing_columns = [  
    column  
    for column in required_columns  
    if column not in rfm.columns  
]  
  
if len(missing_columns) > 0:  
  
    print(  
        "\nMissing Required Columns:"  
    )
```

```
print(missing_columns)
```

```
else:
```

```
print(  
    "\nAll required columns are available."  
)
```

```
# =====  
# STEP 6: CHECK DATA TYPES  
# =====
```

```
print("\nData Types:")  
print(rfm.dtypes)
```

```
# =====  
# STEP 7: CONVERT NUMERICAL COLUMNS  
# =====
```

```
numeric_columns = [  
    "recency",  
    "frequency",  
    "monetary",  
    "R_Score",  
    "F_Score",  
    "M_Score",
```

```
"RFM_Score",  
"churn"
```

```
]
```

```
for column in numeric_columns:
```

```
    if column in rfm.columns:
```

```
        rfm[column] = pd.to_numeric(  
            rfm[column],  
            errors="coerce"  
        )
```

```
# =====  
# STEP 8: CHECK MISSING VALUES  
# =====
```

```
print("\nMissing Values:")
```

```
missing_values = (  
    rfm.isnull()  
    .sum()  
    .sort_values(  
        ascending=False  
    )  
)
```

```
display(  
    missing_values[  
        missing_values > 0  
    ]  
)  
  
# =====  
# STEP 9: REMOVE INVALID CHURN VALUES  
# =====  
  
# Churn should contain only 0 and 1  
  
if "churn" in rfm.columns:  
  
    rfm = rfm[  
        rfm["churn"].isin([0, 1])  
    ].copy()  
  
print(  
    "\nDataset Shape After Churn Validation:",  
    rfm.shape  
)
```

```
# =====  
# STEP 10: REMOVE INVALID RFM VALUES  
# =====
```

```
# RFM values should not be negative
```

```
rfm = rfm[  
    (rfm["recency"] >= 0) &  
    (rfm["frequency"] > 0) &  
    (rfm["monetary"] > 0)  
].copy()  
  
print(  
    "\nDataset Shape After RFM Validation:",  
    rfm.shape  
)
```

```
# =====  
# STEP 11: CHECK DUPLICATE CUSTOMERS  
# =====
```

```
duplicate_customers = (  
    rfm["customer_id"]  
    .duplicated()  
    .sum()  
)
```

```
print(  
    "\nDuplicate Customer IDs:",  
    duplicate_customers  
)
```

```
# Remove duplicate customer records if any  
if duplicate_customers > 0:
```

```
    rfm = (  
        rfm  
        .drop_duplicates(  
            subset="customer_id",  
            keep="first"  
        )  
        .copy()  
    )
```

```
# =====  
# STEP 12: BASIC RFM STATISTICS  
# =====
```

```
print("\nRFM Descriptive Statistics:")
```

```
display(  
    rfm[  
        [  
            "recency",  
            "frequency",  
            "monetary",  
            "RFM_Score"  
        ]  
    ].describe()  
)
```

```
# =====  
# STEP 13: OVERALL CHURN COUNT  
# =====
```

```
churn_count = (  
    rfm["churn"]  
    .value_counts()  
    .sort_index()  
)  
|
```

```
print("\nChurn Count:")

display(
    churn_count
)

# =====
# STEP 14: OVERALL CHURN RATE
# =====

total_customers = len(rfm)

churned_customers = int(
    rfm["churn"].sum()
)

non_churned_customers = (
    total_customers
    - churned_customers
)

overall_churn_rate = (
    churned_customers
    / total_customers
    * 100
)
```

```
print(  
    "\nTotal Customers:",  
    total_customers  
)  
  
print(  
    "Churned Customers:",  
    churned_customers  
)  
  
print(  
    "Non-Churned Customers:",  
    non_churned_customers  
)  
  
print(  
    "Overall Churn Rate:",  
    round(  
        overall_churn_rate,  
        2  
    ),  
    "%"  
)
```

```
# =====  
# STEP 15: CREATE CHURN SUMMARY TABLE  
# =====  
  
churn_summary = pd.DataFrame({  
  
    "Churn_Status": [  
        "Not Churned",  
        "Churned"  
    ],  
  
    "Customer_Count": [  
        non_churned_customers,  
        churned_customers  
    ]  
})  
  
churn_summary["Percentage"] = (  
    churn_summary["Customer_Count"]  
    / total_customers  
    * 100  
)  
  
churn_summary["Percentage"] = (  
    churn_summary["Percentage"]  
    .round(2)  
)  
  
print("\nChurn Summary:")
```

```
display(  
    churn_summary  
)
```

```
# =====  
# STEP 16: CHURN DISTRIBUTION VISUALIZATION  
# =====
```

```
plt.figure(  
    figsize=(6,3)  
)
```

```
sns.barplot(  
    data=churn_summary,  
    x="Churn_Status",  
    y="Customer_Count"  
)
```

```
plt.title(  
    "Customer Churn Distribution"  
)
```

```
plt.xlabel(  
    "Churn Status"  
)
```

```
plt.ylabel(  
    "Customer Count"
```

```
"Number of Customers"
```

```
)  
  
plt.tight_layout()
```

```
plt.show()
```

```
# =====  
# STEP 17: RFM VALUES BY CHURN STATUS  
# =====
```

```
rfm_churn_comparison = (  
    rfm  
    .groupby("churn")  
    .agg(  
  
        Customers=(  
            "customer_id",  
            "count"  
        ),  
  
        Average_Recency=(  
            "recency",  
            "mean"  
        ),  
  
        Average_Frequency=(  
            "frequency",
```

```

        "mean"
    ),

    Average_Frequency=(
        "frequency",
        "mean"
    ),

    Average_Monetary=(
        "monetary",
        "mean"
    ),

    Average_RFM_Score=(
        "RFM_Score",
        "mean"
    )
)
.reset_index()
)

```

```

rfm_churn_comparison[
    "Churn_Status"
] = (
    rfm_churn_comparison[
        "churn"
    ]
    .map({
        0: "Not Churned",

```

```

        1: "Churned"
    })
)

rfm_churn_comparison = (
    rfm_churn_comparison
    .round(2)
)

print(
    "\nRFM Comparison by Churn Status:"
)

display(
    rfm_churn_comparison
)

# =====
# STEP 18: RECENCY VS CHURN
# =====

recency_churn = (
    rfm
    .groupby("churn")
    .agg(

        Customers=(

```

```
        "customer_id",  
        "count"  
    ),  

```

```
Average_Recency=(  
    "recency",  
    "mean"  
),  

```

```
Median_Recency=(  
    "recency",  
    "median"  
)  

```

```
)  
.reset_index()  

```

```
)
```

```
recency_churn[  
    "Churn_Status"
```

```
] = [
```

```
    recency_churn[  
        "churn"
```

```
]
```

```
.map({  
    0: "Not Churned",  
    1: "Churned"  
})
```

```
]
```

```
print(
    "\nRecency vs Churn:"
)

display(
    recency_churn.round(2)
)

# -----
# RECENCY BOXPLOT
# -----

plt.figure(
    figsize=(6,3)
)

sns.boxplot(
    data=rfm,
    x="churn",
    y="recency"
)

plt.title(
    "Recency by Churn Status"
)

plt.xlabel(
```

```
        "Churn Status"
    )

plt.ylabel(
    "Recency (Days)"
)

plt.xticks(
    [0, 1],
    [
        "Not Churned",
        "Churned"
    ]
)

plt.tight_layout()

plt.show()
```

```
# =====
# STEP 19: FREQUENCY VS CHURN
# =====
```

```
frequency_churn = (
    rfm
    .groupby("churn")
    .agg(|
```

```
Customers=(  
    "customer_id",  
    "count"  
)  
  
Average_Frequency=(  
    "frequency",  
    "mean"  
)  
  
Median_Frequency=(  
    "frequency",  
    "median"  
)  
)  
.reset_index()  
)
```

```
frequency_churn[  
    "Churn_Status"  
] = (  
    frequency_churn[  
        "churn"  
    ]  
    .map({  
        0: "Not Churned",  
        1: "Churned"  
    })  
)
```

```
)

print(
    "\nFrequency vs Churn:"
)

display(
    frequency_churn.round(2)
)

# -----
# FREQUENCY BOXPLOT
# -----

plt.figure(
    figsize=(6,3)
)

sns.boxplot(
    data=rfm,
    x="churn",
    y="frequency"
)

plt.title(
    "Purchase Frequency by Churn Status"
)
```

```
plt.xlabel(  
    "Churn Status"  
)  
  
plt.ylabel(  
    "Purchase Frequency"  
)  
  
plt.xticks(  
    [0, 1],  
    [  
        "Not Churned",  
        "Churned"  
    ]  
)  
  
plt.tight_layout()  
  
plt.show()
```

```
# =====  
# STEP 20: MONETARY VALUE VS CHURN  
# =====
```

```
monetary_churn = (  
    rfm  
    .groupby("churn")  
    .agg(  
        #
```

```

Customers=(
    "customer_id",
    "count"
),

Average_Spending=(
    "monetary",
    "mean"
),

Median_Spending=(
    "monetary",
    "median"
)
)
.reset_index()
)

monetary_churn[
    "Churn_Status"
] = (
    monetary_churn[
        "churn"
    ]
    .map({
        0: "Not Churned",
        1: "Churned"
    })
)

```

```
)

print(
    "\nMonetary Value vs Churn:"
)

display(
    monetary_churn.round(2)
)

# -----
# MONETARY BOXPLOT
# -----

plt.figure(
    figsize=(6,3)
)

sns.boxplot(
    data=rfm,
    x="churn",
    y="monetary"
)

plt.title(
    "Customer Spending by Churn Status"
)
```



```
.agg(  
  
    Customers=(  
        "customer_id",  
        "count"  
    ),  
  
    Average_RFM_Score=(  
        "RFM_Score",  
        "mean"  
    ),  
  
    Median_RFM_Score=(  
        "RFM_Score",  
        "median"  
    )  
)  
.reset_index()  
)
```

```
rfm_score_churn[  
    "Churn_Status"  
] = (  
    rfm_score_churn[  
        "churn"  
    ]  
    .map({  
        0: "Not Churned",  
        1: "Churned"|
```

```
    })  
)  
  
print(  
    "\nRFM Score vs Churn:"  
)  
  
display(  
    rfm_score_churn.round(2)  
)  
  
# -----  
# RFM SCORE BOXPLOT  
# -----  
  
plt.figure(  
    figsize=(6,3)  
)  
  
sns.boxplot(  
    data=rfm,  
    x="churn",  
    y="RFM_Score"  
)  
  
plt.title(  
    "RFM Score by Churn |Status"
```

```
)  
  
plt.xlabel(  
    "Churn Status"  
)  
  
plt.ylabel(  
    "RFM Score"  
)  
  
plt.xticks(  
    [0, 1],  
    [  
        "Not Churned",  
        "Churned"  
    ]  
)  
  
plt.tight_layout()  
  
plt.show()
```

```
# =====  
# STEP 22: CHURN RATE BY RFM SCORE  
# =====  
  
churn_by_rfm_score = (  
    rfm  
    .groupby("RFM_Score")  
    .agg(  
  
        Customers=(  
            "customer_id",  
            "count"  
        ),  
  
        Churned_Customers=(  
            "churn",  
            "sum"  
        ),  
  
        Churn_Rate=(  
            "churn",  
            "mean"  
        )  
    )  
    .reset_index()  
)  
  
churn_by_rfm_score[  
    "Churn_Rate"
```

```

    churn_by_rfm_score[
        "Churn_Rate"
    ] * 100
)

churn_by_rfm_score = (
    churn_by_rfm_score
    .round(2)
)

print(
    "\nChurn Rate by RFM Score:"
)

display(
    churn_by_rfm_score
)

# -----
# RFM SCORE CHURN TREND
# -----

plt.figure(
    figsize=(8,4)
)
|

```

```
sns.lineplot(  
    data=churn_by_rfm_score,  
    x="RFM_Score",  
    y="Churn_Rate",  
    marker="o"  
)  
  
plt.title(  
    "Churn Rate by RFM Score"  
)  
  
plt.xlabel(  
    "RFM Score"  
)  
  
plt.ylabel(  
    "Churn Rate (%)"  
)  
  
plt.tight_layout()  
  
plt.show()
```

```
# =====  
# STEP 23: CHURN BY CUSTOMER SEGMENT  
# =====
```

```
if "Customer_Segment" in rfm.columns:
```

```
    segment_churn = (  
        rfm  
        .groupby("Customer_Segment")  
        .agg(  

```

```
            Customers=(  
                "customer_id",  
                "count"  
            ),  

```

```
            Churned_Customers=(  
                "churn",  
                "sum"  
            ),  

```

```
            Churn_Rate=(  
                "churn",  
                "mean"  
            ),  

```

```
            Average_Spending=(  
                "monetary",  

```

```

        "mean"
    ),

    Average_Frequency=(
        "frequency",
        "mean"
    ),

    Average_Recency=(
        "recency",
        "mean"
    )
)
.reset_index()
)

```

```

segment_churn[
    "Churn_Rate"
] = (
    segment_churn[
        "Churn_Rate"
    ] * 100
)

```

```

segment_churn = (
    segment_churn
    .round(2)
    .sort_values(

```

```
        "Churn_Rate",
        ascending=False
    )

print(
    "\nChurn by Customer Segment:"
)

display(
    segment_churn
)
```

else:

```
print(
    "\nCustomer_Segment column not available."
)

segment_churn = pd.DataFrame()
```

```
# -----
# SEGMENT CHURN VISUALIZATION
# -----
```

if not segment_churn.empty:

```
plt.figure(|
```

```
figsize=(6,3)
)

sns.barplot(
    data=segment_churn,
    x="Churn_Rate",
    y="Customer_Segment"
)

plt.title(
    "Churn Rate by Customer Segment"
)

plt.xlabel(
    "Churn Rate (%)"
)

plt.ylabel(
    "Customer Segment"
)

plt.tight_layout()

plt.show()
```

```
# =====  
# STEP 24: CHURN BY RECENCY GROUP  
# =====  
  
# Create recency groups  
  
rfm["Recency_Group"] = pd.cut(  
  
    rfm["recency"],  
  
    bins=[  
        -1,  
        30,  
        60,  
        90,  
        180,  
        365,  
        np.inf  
    ],  
  
    labels=[  
        "0-30 Days",  
        "31-60 Days",  
        "61-90 Days",  
        "91-180 Days",  
        "181-365 Days",  
        "365+ Days"  
    ]  
)
```

```
# Calculate churn
```

```
recency_group_churn = (  
    rfm  
    .groupby(  
        "Recency_Group",  
        observed=True  
    )  
    .agg(  
        Customers=(  
            "customer_id",  
            "count"  
        ),  
        Churned_Customers=(  
            "churn",  
            "sum"  
        ),  
        Churn_Rate=(  
            "churn",  
            "mean"  
        )  
    )  
    .reset_index()  
)  
recency_group_churn[
```

```

    "Churn_Rate"
] = (
    recency_group_churn[
        "Churn_Rate"
    ] * 100
)

recency_group_churn = (
    recency_group_churn
    .round(2)
)

print(
    "\nChurn by Recency Group:"
)

display(
    recency_group_churn
)

# -----
# RECENCY GROUP VISUALIZATION
# -----

plt.figure(
    figsize=(6,3)
)

```

```
sns.barplot(  
    data=recency_group_churn,  
    x="Recency_Group",  
    y="Churn_Rate"  
)  
  
plt.title(  
    "Churn Rate by Recency Group"  
)  
  
plt.xlabel(  
    "Recency Group"  
)  
  
plt.ylabel(  
    "Churn Rate (%)"  
)  
  
plt.xticks(  
    rotation=30  
)  
  
plt.tight_layout()  
  
plt.show()
```

```
# =====  
# STEP 25: CHURN BY FREQUENCY GROUP  
# =====  
  
# Create frequency groups  
  
rfm["Frequency_Group"] = pd.cut(  
  
    rfm["frequency"],  
  
    bins=[  
        0,  
        1,  
        2,  
        5,  
        10,  
        np.inf  
    ],  
  
    labels=[  
        "1 Purchase",  
        "2 Purchases",  
        "3-5 Purchases",  
        "6-10 Purchases",  
        "10+ Purchases"  
    ]  
)
```

```
# Calculate churn
```

```
frequency_group_churn = (  
    rfm  
    .groupby(  
        "Frequency_Group",  
        observed=True  
    )  
    .agg(  
  
        Customers=(  
            "customer_id",  
            "count"  
        ),  
  
        Churned_Customers=(  
            "churn",  
            "sum"  
        ),  
  
        Churn_Rate=(  
            "churn",  
            "mean"  
        )  
    )  
    .reset_index()  
)
```

```
frequency_group_churn[
    "Churn_Rate"
] = (
    frequency_group_churn[
        "Churn_Rate"
    ] * 100
)

frequency_group_churn = (
    frequency_group_churn
    .round(2)
)

print(
    "\nChurn by Frequency Group:"
)

display(
    frequency_group_churn
)

# -----
# FREQUENCY GROUP VISUALIZATION
# -----

plt.figure(
    figsize=(6,3)
```

```
fig=plt.figure(figsize=(10,5))

sns.barplot(
    data=frequency_group_churn,
    x="Frequency_Group",
    y="Churn_Rate"
)

plt.title(
    "Churn Rate by Purchase Frequency"
)

plt.xlabel(
    "Purchase Frequency Group"
)

plt.ylabel(
    "Churn Rate (%)"
)

plt.xticks(
    rotation=30
)

plt.tight_layout()

plt.show()
|
```

```

# =====
# STEP 26: CHURN BY MONETARY GROUP
# =====

# Create five spending groups using ranking.
# Ranking prevents qcut errors caused by tied values.

rfm["Monetary_Group"] = pd.qcut(

    rfm["monetary"].rank(
        method="first"
    ),

    q=5,

    labels=[
        "Very Low",
        "Low",
        "Medium",
        "High",
        "Very High"
    ]
)

# Calculate churn

monetary_group_churn = (
    rfm

```

```

.groupby(
    "Monetary_Group",
    observed=True
)
.agg(

    Customers=(
        "customer_id",
        "count"
    ),

    Churned_Customers=(
        "churn",
        "sum"
    ),

    Churn_Rate=(
        "churn",
        "mean"
    )
)
.reset_index()
)

```

```

monetary_group_churn[
    "Churn_Rate"
] = (
    monetary_group_churn[

```

```

        "Churn_Rate"
    ] * 100
)

monetary_group_churn = (
    monetary_group_churn
    .round(2)
)

print(
    "\nChurn by Monetary Group:"
)

display(
    monetary_group_churn
)

# -----
# MONETARY GROUP VISUALIZATION
# -----

plt.figure(
    figsize=(6,3)
)

sns.barplot(
    data=monetary_group_churn,
    x="Monetary_Group",

```

```

    y="Churn_Rate"
)

plt.title(
    "Churn Rate by Customer Spending Group"
)

plt.xlabel(
    "Spending Group"
)

plt.ylabel(
    "Churn Rate (%)"
)

plt.tight_layout()

plt.show()

# =====
# STEP 27: CHURN BY AGE GROUP
# =====

if "customer_age" in rfm.columns:

    # Create age groups

    rfm["Age_Group"] = pd.cut(

```

```
rfm["customer_age"],
```

```
bins=[  
    0,  
    18,  
    25,  
    35,  
    45,  
    55,  
    65,  
    100
```

```
],
```

```
labels=[  
    "Under 18",  
    "18-25",  
    "26-35",  
    "36-45",  
    "46-55",  
    "56-65",  
    "65+"
```

```
],
```

```
include_lowest=True
```

```
)
```

```
# Convert to percentage
```

```
age_churn[  
    "Churn_Rate"  
] = (  
    age_churn[  
        "Churn_Rate"  
    ] * 100  
)
```

```
age_churn = (  
    age_churn  
    .round(2)  
)
```

```
print(  
    "\nChurn by Age Group:"  
)
```

```
display(  
    age_churn  
)
```

```
# Visualization
```

```
plt.figure(  
    figsize=(6,3)  
)
```

```
sns.barplot(  
    data=age_churn,  
    x="Age_Group",  
    y="Churn_Rate"  
)
```

```
plt.title(  
    "Churn Rate by Age Group"  
)
```

```
plt.xlabel(  
    "Age Group"  
)
```

```
plt.ylabel(  
    "Churn Rate (%)"  
)
```

```
plt.xticks(  
    rotation=30  
)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
else:
```

```
print(  
    "\ncustomer_age column is not available."  
)
```

```
age_churn = pd.DataFrame()
```

```
# =====  
# STEP 28: CHURN BY GENDER  
# =====
```

```
if "gender" in rfm.columns:
```

```
gender_churn = (  
    rfm  
    .groupby("gender")  
    .agg(  
  
        Customers=(  
            "customer_id",  
            "count"  
        ),  
    ),
```

```
    Churned_Customers=(
        "churn",
        "sum"
    ),

    Churn_Rate=(
        "churn",
        "mean"
    )
)
.reset_index()

)

# Convert churn rate to percentage
```

```
gender_churn[
    "Churn_Rate"
] = (
    gender_churn[
        "Churn_Rate"
    ] * 100
)
```

```
gender_churn = (
    gender_churn
    .round(2)
)
```

```
)

print(
    "\nChurn by Gender:"
)

display(
    gender_churn
)

# Visualization

plt.figure(
    figsize=(6,3)
)

sns.barplot(
    data=gender_churn,
    x="gender",
    y="Churn_Rate"
)

plt.title(
    "Churn Rate by Gender"
)
```

```
plt.xlabel(  
    "Gender"  
)  
  
plt.ylabel(  
    "Churn Rate (%)"  
)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
else:
```

```
print(  
    "\ngender column is not available."  
)
```

```
gender_churn = pd.DataFrame()
```

```

# =====
# STEP 29: RFM SCORE COMPONENT ANALYSIS
# =====

# -----
# RECENCY SCORE VS CHURN
# -----

r_score_churn = (
    rfm
    .groupby("R_Score")
    .agg(

        Customers=(
            "customer_id",
            "count"
        ),

        Churned_Customers=(
            "churn",
            "sum"
        ),

        Churn_Rate=(
            "churn",
            "mean"
        )
    )
)

```

```

        .reset_index()
    )

r_score_churn[
    "Churn_Rate"
] *= 100

r_score_churn = (
    r_score_churn
    .round(2)
)

print(
    "\nChurn by Recency Score:"
)

display(
    r_score_churn
)

# -----
# FREQUENCY SCORE VS CHURN
# -----

f_score_churn = (
    rfm
    | .groupby("F_Score")

```

```
.agg(
```

```
    Customers=(  
        "customer_id",  
        "count"  
    ),
```

```
    Churned_Customers=(  
        "churn",  
        "sum"  
    ),
```

```
    Churn_Rate=(  
        "churn",  
        "mean"  
    )
```

```
)  
.reset_index()
```

```
)
```

```
f_score_churn[  
    "Churn_Rate"  
] *= 100
```

```
f_score_churn = (  
    f_score_churn  
    .round(2)
```

```
)
```

```
print(  
    "\nChurn by Frequency Score:"  
)
```

```
display(  
    f_score_churn  
)
```

```
# -----  
# MONETARY SCORE VS CHURN  
# -----
```

```
m_score_churn = (  
    rfm  
    .groupby("M_Score")  
    .agg(  
  
        Customers=(  
            "customer_id",  
            "count"  
        ),  
  
        Churned_Customers=(  
            "churn",  
            "sum"  
        ),  
    )
```

```
        Churn_Rate=(  
            "churn",  
            "mean"  
        )  
    )  
    .reset_index()  
)  
  
m_score_churn[  
    "Churn_Rate"  
] *= 100  
  
m_score_churn = (  
    m_score_churn  
    .round(2)  
)  
  
print(  
    "\nChurn by Monetary Score:"  
)  
  
display(  
    m_score_churn  
)
```

```
# =====  
# STEP 30: RFM CHURN HEATMAP  
# =====  
  
rfm_churn_heatmap = (  
    rfm  
    .pivot_table(  
        index="R_Score",  
        columns="F_Score",  
        values="churn",  
        aggfunc="mean"  
    )  
)  
  
# Convert to percentage  
  
rfm_churn_heatmap = (  
    rfm_churn_heatmap * 100  
)  
  
print(  
    "\nRFM Churn Heatmap Data:"  
)  
  
display(  
    rfm_churn_heatmap.round(2)  
)  
  
|
```

```
plt.figure(  
    figsize=(6,3)  
)  
  
sns.heatmap(  
    rfm_churn_heatmap,  
    annot=True,  
    fmt=".1f"  
)  
  
plt.title(  
    "Churn Rate (%) by Recency Score and Frequency Score"  
)  
  
plt.xlabel(  
    "Frequency Score"  
)  
  
plt.ylabel(  
    "Recency Score"  
)  
  
plt.tight_layout()  
  
plt.show()
```

```
# =====  
# STEP 31: IDENTIFY HIGH-RISK CUSTOMERS  
# =====  
  
# High-risk conditions:  
#  
# 1. Already churned  
# OR  
# 2. Very low recency score  
# OR  
# 3. Very low overall RFM score  
  
high_risk_customers = rfm[  
    (  
        (rfm["churn"] == 1)  
        |  
        (rfm["R_Score"] <= 2)  
        |  
        (rfm["RFM_Score"] <= 6)  
    )  
].copy()  
  
print(  
    "\nHigh-Risk Customers:",  
    len(high_risk_customers)  
)
```

```
# Display high-risk customers
```

```
display(  
  high_risk_customers[  
    [  
      "customer_id",  
      "recency",  
      "frequency",  
      "monetary",  
      "R_Score",  
      "F_Score",  
      "M_Score",  
      "RFM_Score",  
      "churn"  
    ]  
  ]  
  .sort_values(  
    "monetary",  
    ascending=False  
  )  
  .head(20)  
)
```

```
# =====  
# STEP 32: HIGH-VALUE AT-RISK CUSTOMERS  
# =====  
  
# High-value customers with poor recency  
  
high_value_at_risk = rfm[(  
    (rfm["R_Score"] <= 2)  
    &  
    (rfm["M_Score"] >= 4)  
    &  
    (rfm["churn"] == 0)  
)  
  
print(  
    "\nHigh-Value At-Risk Customers:",  
    len(high_value_at_risk)  
)  
  
display(  
    high_value_at_risk[  
        [  
            "customer_id",  
            "recency",  
            "frequency",  
            ]
```

```
        "monetary",
        "R_Score",
        "F_Score",
        "M_Score",
        "RFM_Score",
        "churn"
    ]
]
.sort_values(
    "monetary",
    ascending=False
)
.head(20)
)
```

```
# =====
# STEP 33: CHURNED HIGH-VALUE CUSTOMERS
# =====
```

```
# Customers who have churned
# and belong to the high monetary-value group
```

```
churned_high_value = rfm[
    (
        (rfm["churn"] == 1)
        &
        (rfm["M_Score"] >= 4)
    )
]
```

```
].copy()

print(
    "\nChurned High-Value Customers:",
    len(churned_high_value)
)

display(
    churned_high_value[
        [
            "customer_id",
            "recency",
            "frequency",
            "monetary",
            "R_Score",
            "F_Score",
            "M_Score",
            "RFM_Score"
        ]
    ]
    .sort_values(
        "monetary",
        ascending=False
    )
    .head(20)
)
```

```
# =====  
# STEP 34: REVENUE IMPACT OF CHURN  
# =====  
  
# Revenue associated with churned customers  
  
churned_revenue = (  
    rfm  
    .loc[  
        rfm["churn"] == 1,  
        "monetary"  
    ]  
    .sum()  
)  
  
# Revenue associated with active customers  
  
active_revenue = (  
    rfm  
    .loc[  
        rfm["churn"] == 0,  
        "monetary"  
    ]  
    .sum()  
)
```

```
# Total revenue
```

```
total_customer_revenue = (  
    rfm["monetary"]  
    .sum()  
)
```

```
# Calculate percentage
```

```
if total_customer_revenue > 0:
```

```
    churned_revenue_percentage = (  
        churned_revenue  
        / total_customer_revenue  
        * 100  
    )
```

```
else:
```

```
    churned_revenue_percentage = 0
```

```
print(  
    "\nChurned Customer Revenue:",  
    round(  
        churned_revenue,  
        2  
    )  
)
```

```
)  
  
print(  
    "Active Customer Revenue:",  
    round(  
        active_revenue,  
        2  
    )  
)  
  
print(  
    "Total Customer Revenue:",  
    round(  
        total_customer_revenue,  
        2  
    )  
)  
  
print(  
    "Churned Revenue Percentage:",  
    round(  
        churned_revenue_percentage,  
        2  
    ),  
    "%"  
)
```

```
# =====  
# STEP 35: CREATE CHURN RISK CATEGORIES  
# =====  
  
def assign_churn_risk(row):  
  
    # Customers who are already churned  
    if row["churn"] == 1:  
  
        return "Churned"  
  
    # High-risk customers  
    elif (  
        row["R_Score"] <= 2  
        and  
        row["F_Score"] <= 2  
    ):  
  
        return "High Risk"  
  
    # Medium-risk customers  
    elif row["R_Score"] <= 2:  
  
        return "Medium Risk"  
  
    # Low-risk customers  
    elif row["R_Score"] >= 4:
```

```
    return "Low Risk"
```

```
# Remaining customers
```

```
else:
```

```
    return "Moderate Risk"
```

```
# Apply function
```

```
rfm["Churn_Risk"] = (  
    rfm.apply(  
        assign_churn_risk,  
        axis=1  
    )  
)
```

```
# =====
```

```
# STEP 36: CHURN RISK SUMMARY
```

```
# =====
```

```
risk_summary = (  
    rfm["Churn_Risk"]  
    .value_counts()  
    .reset_index()  
)
```

```
|
```

```
risk_summary.columns = [  
    "Churn_Risk",  
    "Customer_Count"  
]  
  
risk_summary["Percentage"] = (  
    risk_summary["Customer_Count"]  
    / len(rfm)  
    * 100  
)  
  
risk_summary["Percentage"] = (  
    risk_summary["Percentage"]  
    .round(2)  
)  
  
print(  
    "\nChurn Risk Summary:"  
)  
  
display(  
    risk_summary  
)
```

```
#####  
# STEP 37: CHURN RISK VISUALIZATION  
# =====  
  
plt.figure(  
    figsize=(6,3)  
)  
  
sns.barplot(  
    data=risk_summary,  
    x="Customer_Count",  
    y="Churn_Risk"  
)  
  
plt.title(  
    "Customer Churn Risk Distribution"  
)  
  
plt.xlabel(  
    "Number of Customers"  
)  
  
plt.ylabel(  
    "Churn Risk"  
)  
  
plt.tight_layout()  
  
plt.show()
```

```
# =====  
# STEP 38: CORRELATION ANALYSIS  
# =====  
  
correlation_columns = [  
    "recency",  
    "frequency",  
    "monetary",  
    "R_Score",  
    "F_Score",  
    "M_Score",  
    "RFM_Score",  
    "churn"  
]  
  
# Keep only available columns  
  
available_columns = [  
    column  
    for column in correlation_columns  
    if column in rfm.columns  
]  
  
# Create correlation matrix  
  
correlation_matrix = (  
    rfm[
```

```
        available_columns
    ]
    .corr()
)

print(
    "\nCorrelation Matrix:"
)

display(
    correlation_matrix.round(2)
)

# =====
# STEP 39: CORRELATION HEATMAP
# =====

plt.figure(
    figsize=(6,3)
)

sns.heatmap(
    correlation_matrix,
    annot=True,
    fmt=".2f"
)
```

```
plt.title(  
    "RFM and Churn Correlation Matrix"  
)
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# =====  
# STEP 40: FINAL CHURN SUMMARY  
# =====
```

```
final_churn_summary = pd.DataFrame({
```

```
    "Metric": [  
        "Total Customers",  
        "Churned Customers",  
        "Non-Churned Customers",  
        "Overall Churn Rate (%)",  
        "Average Recency - Churned",  
        "Average Recency - Non-Churned",
```

```
"Overall Churn Rate (%)",  
  
"Average Recency - Churned",  
  
"Average Recency - Non-Churned",  
  
"Average Frequency - Churned",  
  
"Average Frequency - Non-Churned",  
  
"Average Spending - Churned",  
  
"Average Spending - Non-Churned",  
  
"Churned Customer Revenue",  
  
"Churned Revenue Percentage (%)",  
  
"High-Risk Customers",  
  
"High-Value At-Risk Customers",  
  
"Churned High-Value Customers"  
],  
  
"Value": [  
  
    total_customers,
```

```
churned_customers,
```

```
non_churned_customers,
```

```
round(  
    overall_churn_rate,  
    2  
)
```

```
round(  
    rfm.loc[  
        rfm["churn"] == 1,  
        "recency"  
    ].mean(),  
    2  
)
```

```
round(  
    rfm.loc[  
        rfm["churn"] == 0,  
        "recency"  
    ].mean(),  
    2  
)
```

```
round(  
    rfm.loc[  
        rfm["churn"] == 1,
```

```
    "frequency"  
].mean(),  
2  
,  
  
round(  
    rfm.loc[  
        rfm["churn"] == 0,  
        "frequency"  
    ].mean(),  
2  
,  
  
round(  
    rfm.loc[  
        rfm["churn"] == 1,  
        "monetary"  
    ].mean(),  
2  
,  
  
round(  
    rfm.loc[  
        rfm["churn"] == 0,  
        "monetary"  
    ].mean(),  
2  
,|
```

```
round(
    churned_revenue,
    2
),

round(
    churned_revenue_percentage,
    2
),

len(
    high_risk_customers
),

len(
    high_value_at_risk
),

len(
    churned_high_value
)
]
})

print(
    "\nFinal Churn Analysis Summary:"
)
```

```
display(
    final_churn_summary
)

# =====
# STEP 41: SAVE COMPLETE CHURN DATASET
# =====

rfm.to_csv(
    "customer_churn_analysis.csv",
    index=False
)

print(
    "Saved: customer_churn_analysis.csv"
)

# =====
# STEP 42: SAVE CHURN SUMMARY
# =====

churn_summary.to_csv(
    "overall_churn_summary.csv",
    index=False
)
```

```
final_churn_summary.to_csv(
    "final_churn_summary.csv",
    index=False
)

print(
    "Saved: overall_churn_summary.csv"
)

print(
    "Saved: final_churn_summary.csv"
)

# =====
# STEP 43: SAVE RFM CHURN ANALYSIS
# =====

rfm_churn_comparison.to_csv(
    "rfm_churn_comparison.csv",
    index=False
)

churn_by_rfm_score.to_csv(
    "rfm_score_churn_analysis.csv",
    index=False
)
```

```
print(
    "Saved: RFM churn analysis files."
)

# =====
# STEP 44: SAVE GROUP ANALYSIS FILES
# =====

recency_group_churn.to_csv(
    "recency_group_churn.csv",
    index=False
)

frequency_group_churn.to_csv(
    "frequency_group_churn.csv",
    index=False
)

monetary_group_churn.to_csv(
    "monetary_group_churn.csv",
    index=False
)

print(
    "Saved: Recency, Frequency and Monetary group files."
)
```

```
# =====  
# STEP 45: SAVE AGE AND GENDER ANALYSIS  
# =====
```

```
if not age_churn.empty:
```

```
    age_churn.to_csv(  
        "age_group_churn.csv",  
        index=False  
    )
```

```
    print(  
        "Saved: age_group_churn.csv"  
    )
```

```
if not gender_churn.empty:
```

```
    gender_churn.to_csv(  
        "gender_churn_analysis.csv",  
        index=False  
    )
```

```
    print(  
        "Saved: gender_churn_analysis.csv"  
    )
```

```
# =====  
# STEP 46: SAVE RFM SCORE COMPONENT ANALYSIS  
# =====  
  
r_score_churn.to_csv(  
    "recency_score_churn.csv",  
    index=False  
)  
  
f_score_churn.to_csv(  
    "frequency_score_churn.csv",  
    index=False  
)  
  
m_score_churn.to_csv(  
    "monetary_score_churn.csv",  
    index=False  
)  
  
print(  
    "Saved: RFM score component files."  
)
```

```
# =====  
# STEP 47: SAVE RFM HEATMAP DATA  
# =====  
  
rfm_churn_heatmap.to_csv(  
    "rfm_churn_heatmap.csv"  
)  
  
print(  
    "Saved: rfm_churn_heatmap.csv"  
)  
  
# =====  
# STEP 48: SAVE SEGMENT CHURN ANALYSIS  
# =====  
  
if not segment_churn.empty:  
  
    segment_churn.to_csv(  
        "segment_churn_analysis.csv",  
        index=False  
    )  
  
    print(  
        "Saved: segment_churn_analysis.csv"  
    )
```

```
# =====  
# STEP 49: SAVE HIGH-RISK CUSTOMER FILES  
# =====
```

```
high_risk_customers.to_csv(  
    "high_risk_customers.csv",  
    index=False  
)  
  
high_value_at_risk.to_csv(  
    "high_value_at_risk_customers.csv",  
    index=False  
)  
  
churned_high_value.to_csv(  
    "churned_high_value_customers.csv",  
    index=False  
)  
  
print(  
    "Saved: High-risk customer files."  
)
```

```
# =====  
# STEP 50: FINAL STEP 5 REPORT  
# =====  
  
print("\n")  
print("=" * 70)  
print("STEP 5: CHURN ANALYSIS COMPLETED SUCCESSFULLY")  
print("=" * 70)  
  
print(  
    "\nTotal Customers:",  
    total_customers  
)  
  
print(  
    "Churned Customers:",  
    churned_customers  
)  
  
print(  
    "Non-Churned Customers:",  
    non_churned_customers  
)  
  
print(  
    "Overall Churn Rate:",  
    round(  
        overall_churn_rate,  
        2
```

```
    ),  
    "%"  
)  
  
print(  
    "High-Risk Customers:",  
    len(high_risk_customers)  
)  
  
print(  
    "High-Value At-Risk Customers:",  
    len(high_value_at_risk)  
)  
  
print(  
    "Churned High-Value Customers:",  
    len(churned_high_value)  
)  
  
print(  
    "Churned Customer Revenue:",  
    round(  
        churned_revenue,  
        2  
    )  
)  
  
print("\n")
```

```
print("\n")
print("=" * 70)
print("STEP 5 OUTPUT FILES")
print("=" * 70)

print("1. customer_churn_analysis.csv")
print("2. overall_churn_summary.csv")
print("3. final_churn_summary.csv")
print("4. rfm_churn_comparison.csv")
print("5. rfm_score_churn_analysis.csv")
print("6. recency_group_churn.csv")
print("7. frequency_group_churn.csv")
print("8. monetary_group_churn.csv")
print("9. age_group_churn.csv")
print("10. gender_churn_analysis.csv")
print("11. recency_score_churn.csv")
print("12. frequency_score_churn.csv")
print("13. monetary_score_churn.csv")
print("14. rfm_churn_heatmap.csv")
print("15. segment_churn_analysis.csv")
print("16. high_risk_customers.csv")
print("17. high_value_at_risk_customers.csv")
print("18. churned_high_value_customers.csv")

print("\nStep 5 completed.")
```

Libraries imported successfully.
RFM dataset loaded successfully.

Dataset Shape:
(49673, 12)

First 5 Records:

	customer_id	customer_name	recency	frequency	monetary	churn	R_Score	F_Score	M_Score	RFM_Score	RFM_Score_Code	Customer_Segment
0	1	Nicole Johnson	57	1	3491	0	5	1	1	7	511	New Customers
1	2	Marie Wright	298	3	7988	0	2	1	2	5	212	Hibernating
2	3	Julie Wolfe	88	8	22587	0	4	5	5	14	455	Champions
3	4	Tracey Smith	126	4	8715	1	4	2	2	8	422	Potential Loyalists
4	5	Nancy Jones	170	8	12524	0	3	5	3	11	353	Loyal Customers

Available Columns:
['customer_id', 'customer_name', 'recency', 'frequency', 'monetary', 'churn', 'R_Score', 'F_Score', 'M_Score', 'RFM_Score', 'RFM_Score_Code', 'Customer_Segment']

Standardized Columns:
['customer_id', 'customer_name', 'recency', 'frequency', 'monetary', 'churn', 'R_Score', 'F_Score', 'M_Score', 'RFM_Score', 'RFM_Score_Code', 'Customer_Segment']

All required columns are available.

All required columns are available.

Data Types:

customer_id	int64
customer_name	object
recency	int64
frequency	int64
monetary	int64
churn	int64
R_Score	int64
F_Score	int64
M_Score	int64
RFM_Score	int64
RFM_Score_Code	int64
Customer_Segment	object
dtype:	object

Missing Values:

Series([], dtype: int64)

Dataset Shape After Churn Validation: (49673, 12)

Dataset Shape After RFM Validation: (49673, 12)

Duplicate Customer IDs: 0

RFM Descriptive Statistics:

	recency	frequency	monetary	RFM_Score
count	49673.000000	49673.000000	49673.000000	49673.000000
mean	260.424979	5.032915	13716.559962	9.000000
std	246.024001	2.206427	6840.208557	3.450266
min	0.000000	1.000000	110.000000	3.000000
25%	77.000000	3.000000	8719.000000	6.000000
50%	184.000000	5.000000	13026.000000	9.000000
75%	368.000000	6.000000	17966.000000	12.000000
max	1352.000000	17.000000	55339.000000	15.000000

Churn Count:

churn

0 39731

1 9942

Name: count, dtype: int64

Total Customers: 49673

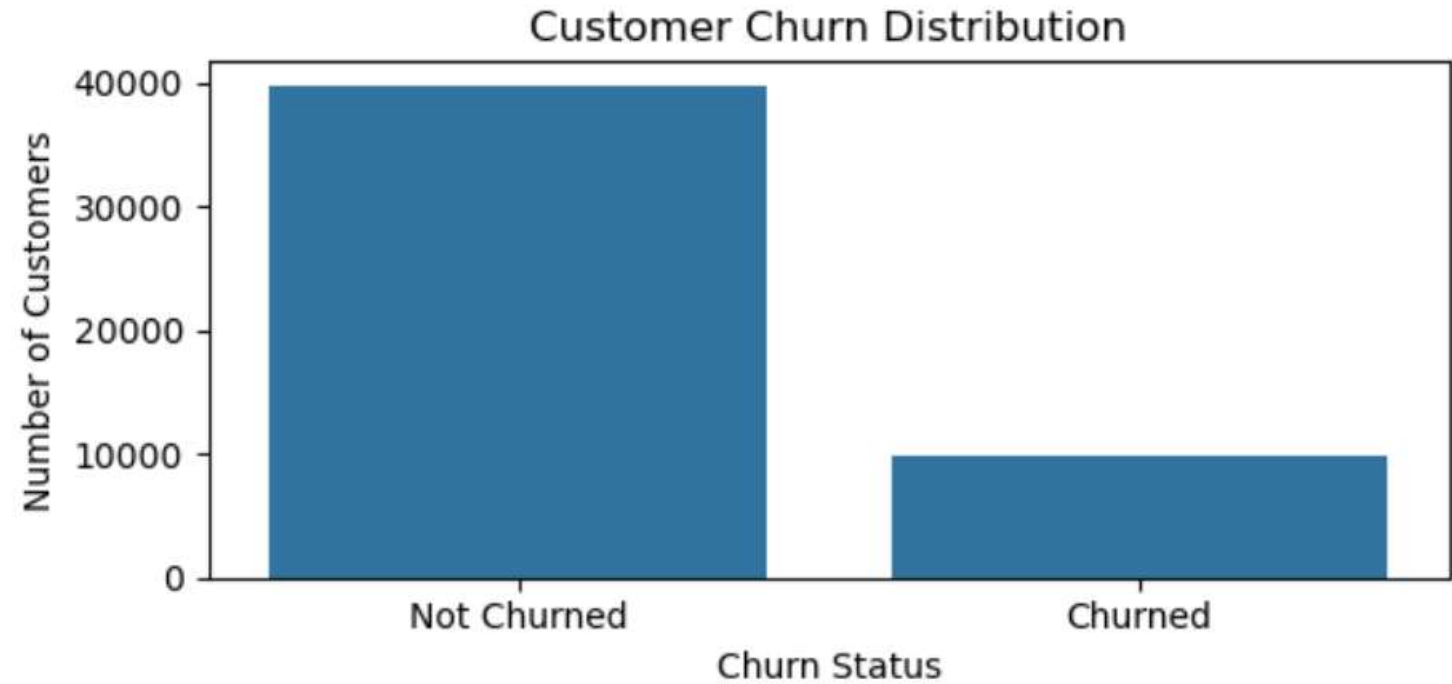
Churned Customers: 9942

Non-Churned Customers: 39731

Overall Churn Rate: 20.01 %

Churn Summary:

	Churn_Status	Customer_Count	Percentage
0	Not Churned	39731	79.99
1	Churned	9942	20.01



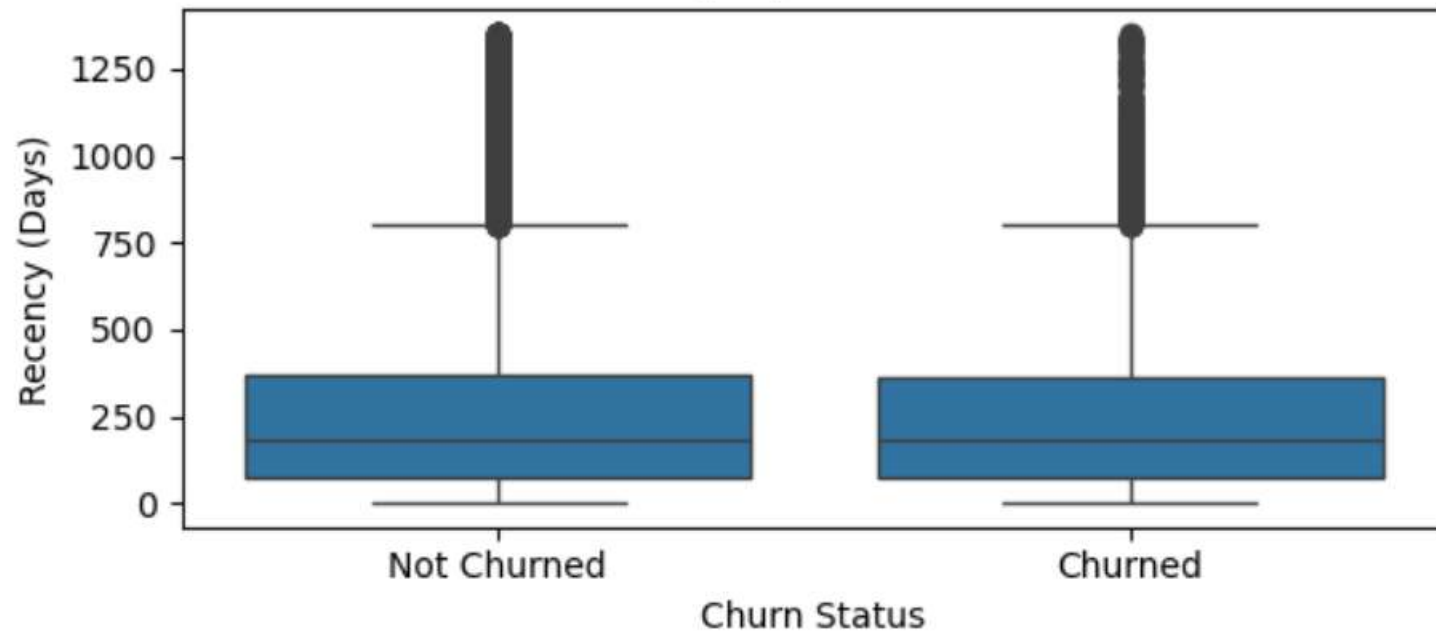
RFM Comparison by Churn Status:

	churn	Customers	Average_Recency	Average_Frequency	Average_Monetary	Average_RFM_Score	Churn_Status
0	0	39731	260.89	5.04	13724.96	9.00	Not Churned
1	1	9942	258.56	5.02	13682.98	8.99	Churned

Recency vs Churn:

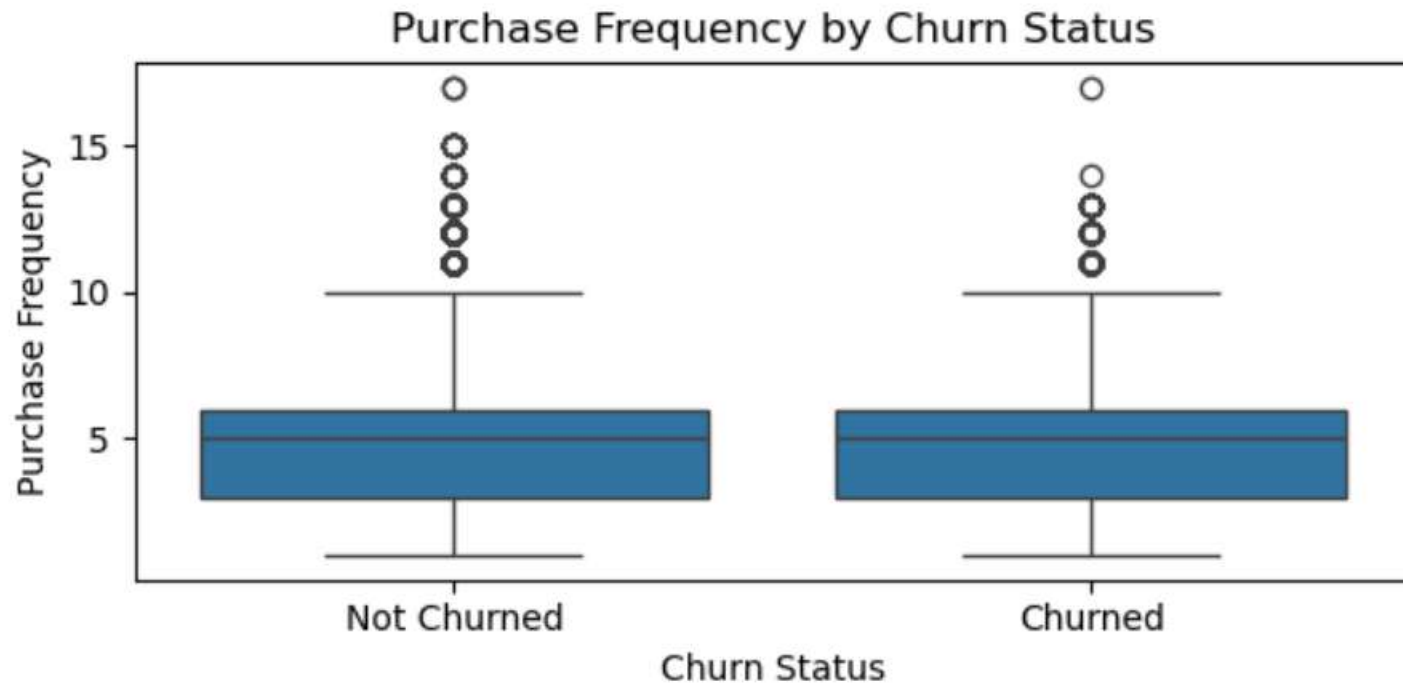
	churn	Customers	Average_Recency	Median_Recency	Churn_Status
0	0	39731	260.89	184.0	Not Churned
1	1	9942	258.56	185.0	Churned

Recency by Churn Status



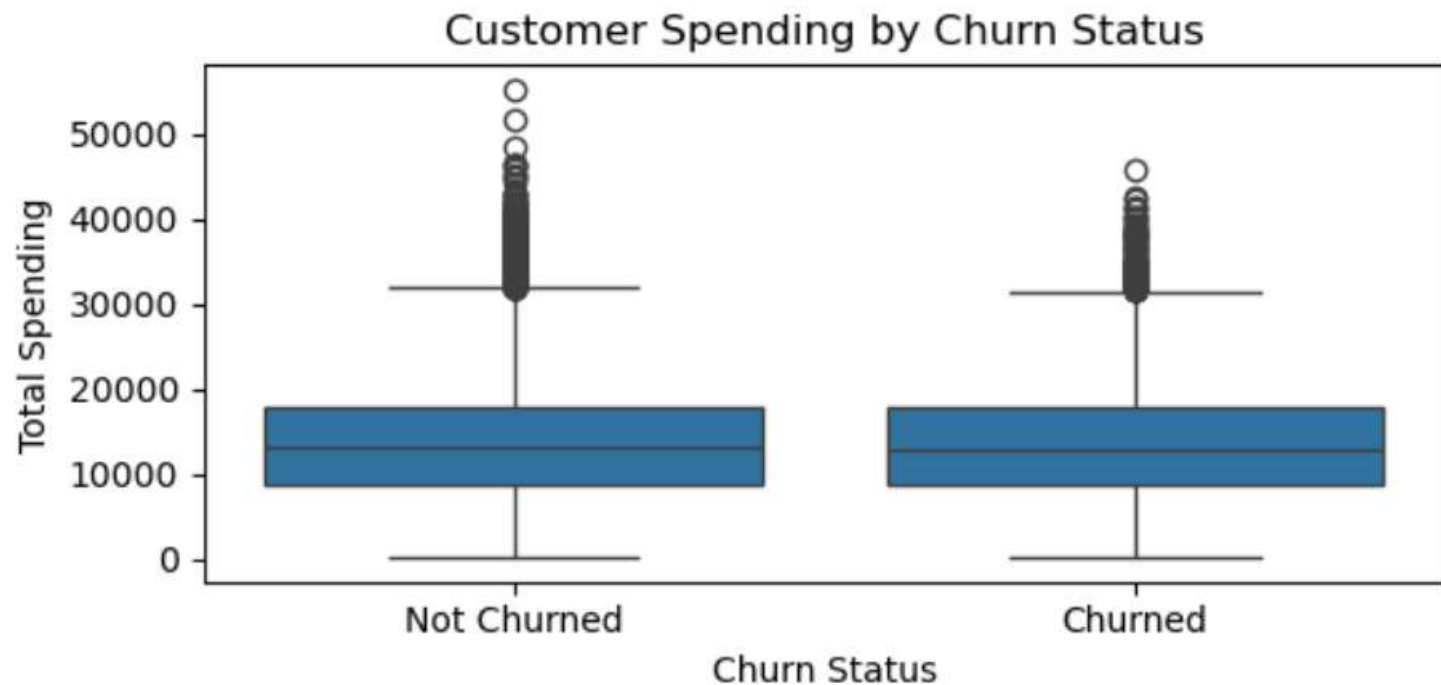
Frequency vs Churn:

	churn	Customers	Average_Frequency	Median_Frequency	Churn_Status
0	0	39731	5.04	5.0	Not Churned
1	1	9942	5.02	5.0	Churned



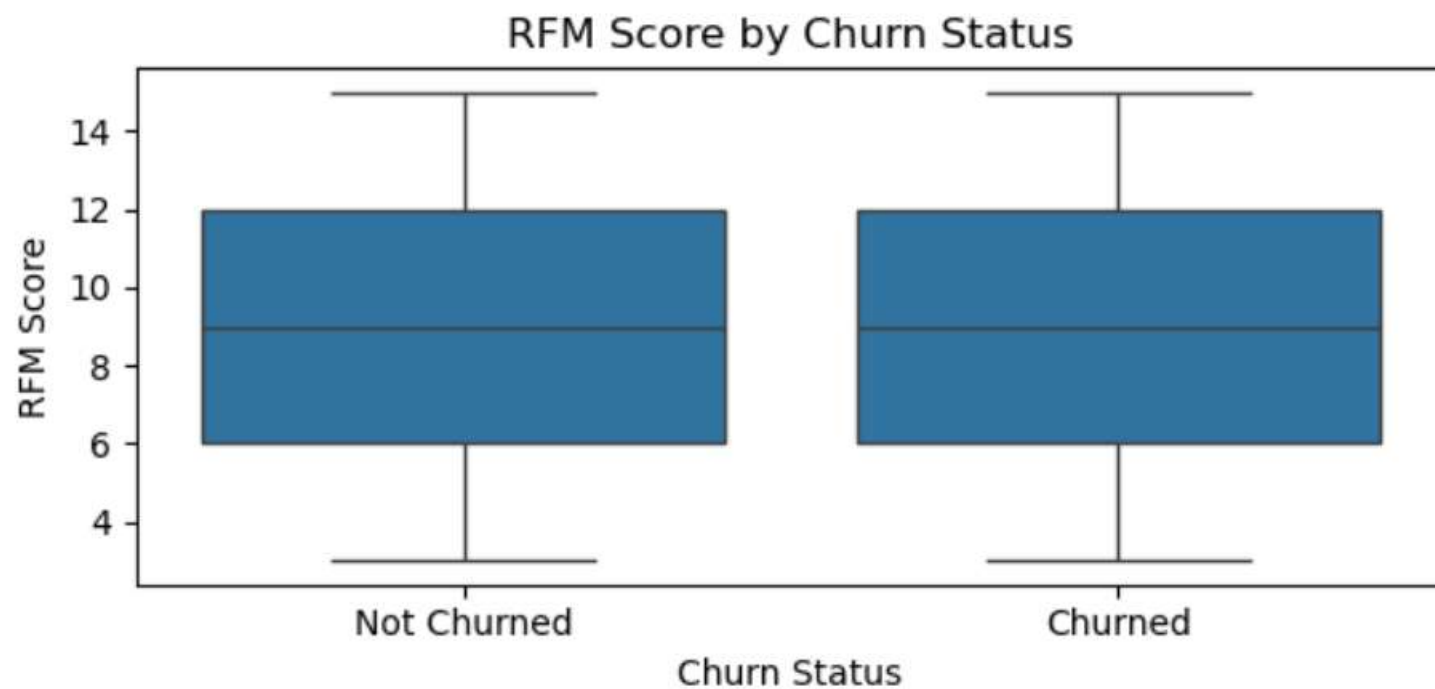
Monetary Value vs Churn:

	churn	Customers	Average_Spending	Median_Spending	Churn_Status
0	0	39731	13724.96	13054.0	Not Churned
1	1	9942	13682.98	12926.0	Churned



RFM Score vs Churn:

	churn	Customers	Average_RFM_Score	Median_RFM_Score	Churn_Status
0	0	39731	9.00	9.0	Not Churned
1	1	9942	8.99	9.0	Churned

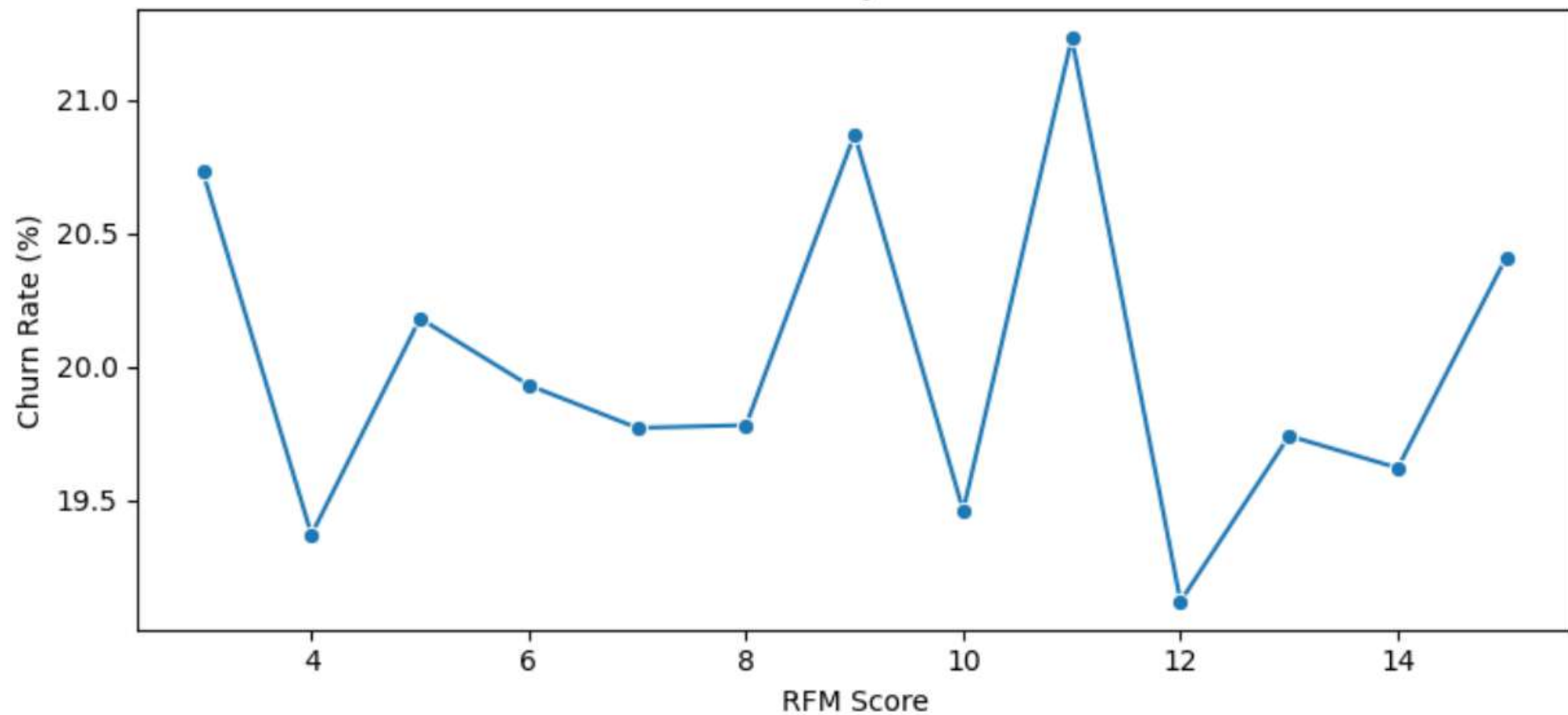


Churn Rate by RFM Score:

	RFM_Score	Customers	Churned_Customers	Churn_Rate
0	3	3396	704	20.73
1	4	2788	540	19.37
2	5	3384	683	20.18
3	6	3824	762	19.93
4	7	4386	867	19.77

5	8	4463	883	19.78
6	9	4470	933	20.87
7	10	4558	887	19.46
8	11	4460	947	21.23
9	12	4498	860	19.12
10	13	3916	773	19.74
11	14	3257	639	19.62
12	15	2273	464	20.41

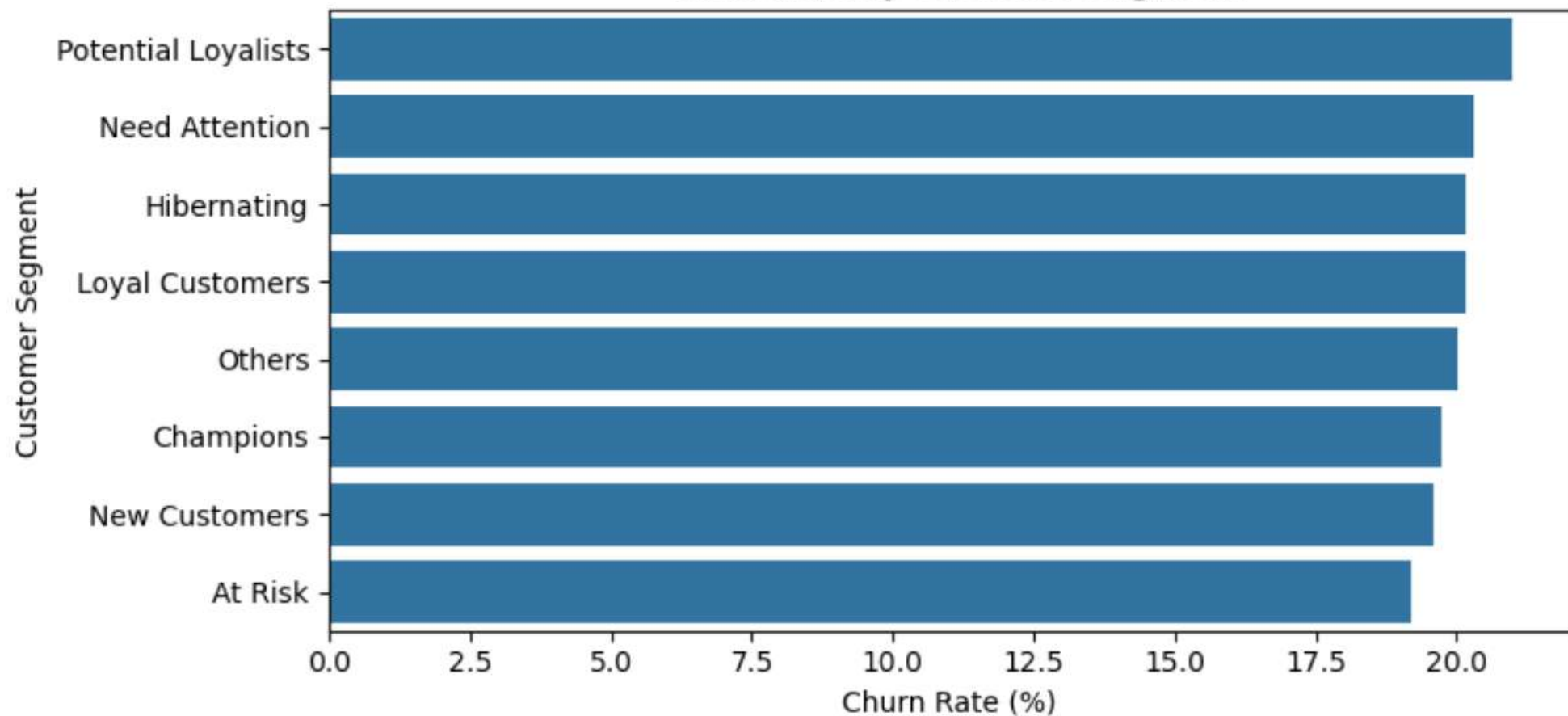
Churn Rate by RFM Score



Churn by Customer Segment:

	Customer_Segment	Customers	Churned_Customers	Churn_Rate	Average_Spending	Average_Frequency	Average_Recency
7	Potential Loyalists	6722	1410	20.98	12507.72	4.53	64.60
4	Need Attention	4074	827	20.30	13901.43	4.19	314.09
2	Hibernating	9347	1883	20.15	6490.42	2.59	578.92
3	Loyal Customers	5644	1137	20.15	18300.67	6.97	153.39
6	Others	5111	1024	20.04	8102.03	3.91	254.12
1	Champions	8692	1714	19.72	21789.39	7.65	59.89
5	New Customers	2812	551	19.59	6505.87	2.71	66.41
0	At Risk	7271	1396	19.20	17546.28	6.16	404.24

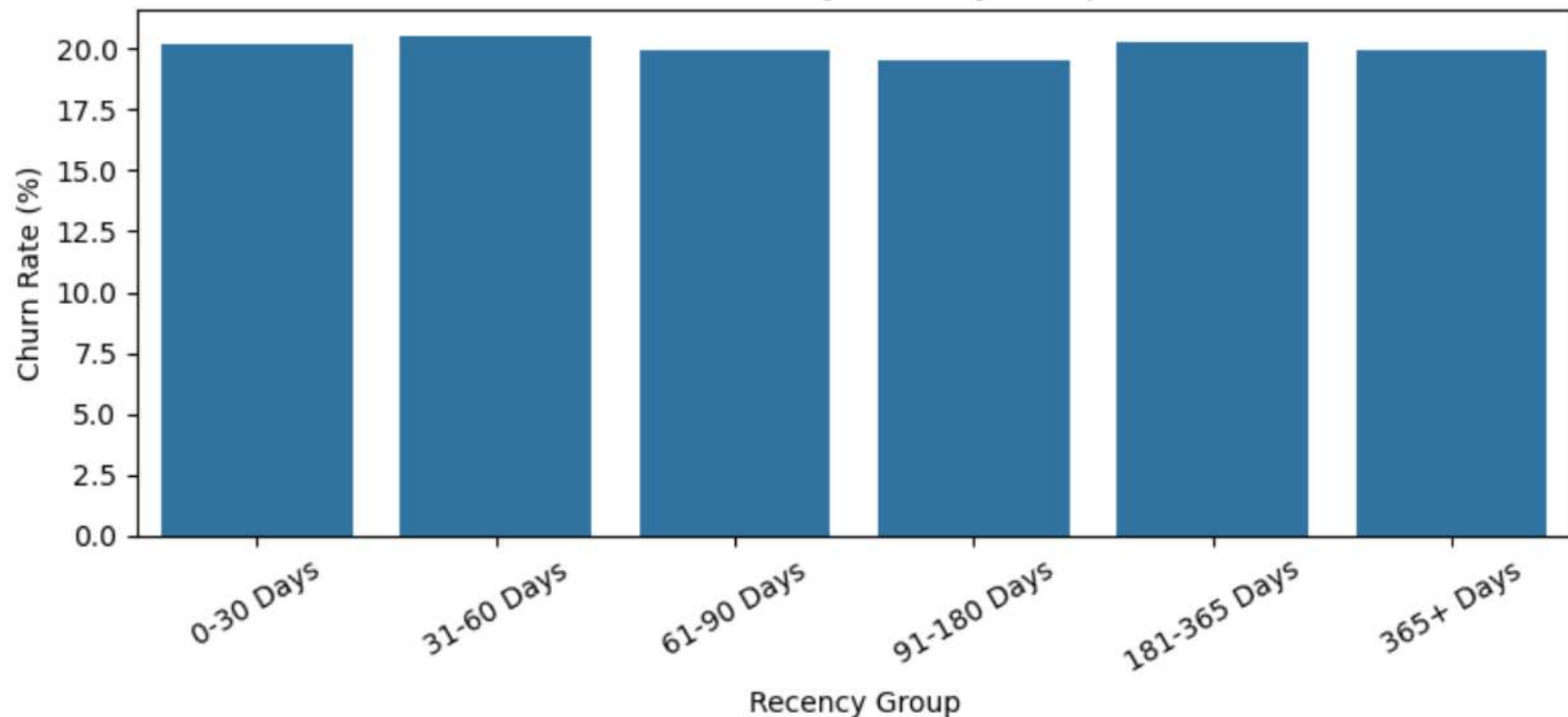
Churn Rate by Customer Segment



Churn by Recency Group:

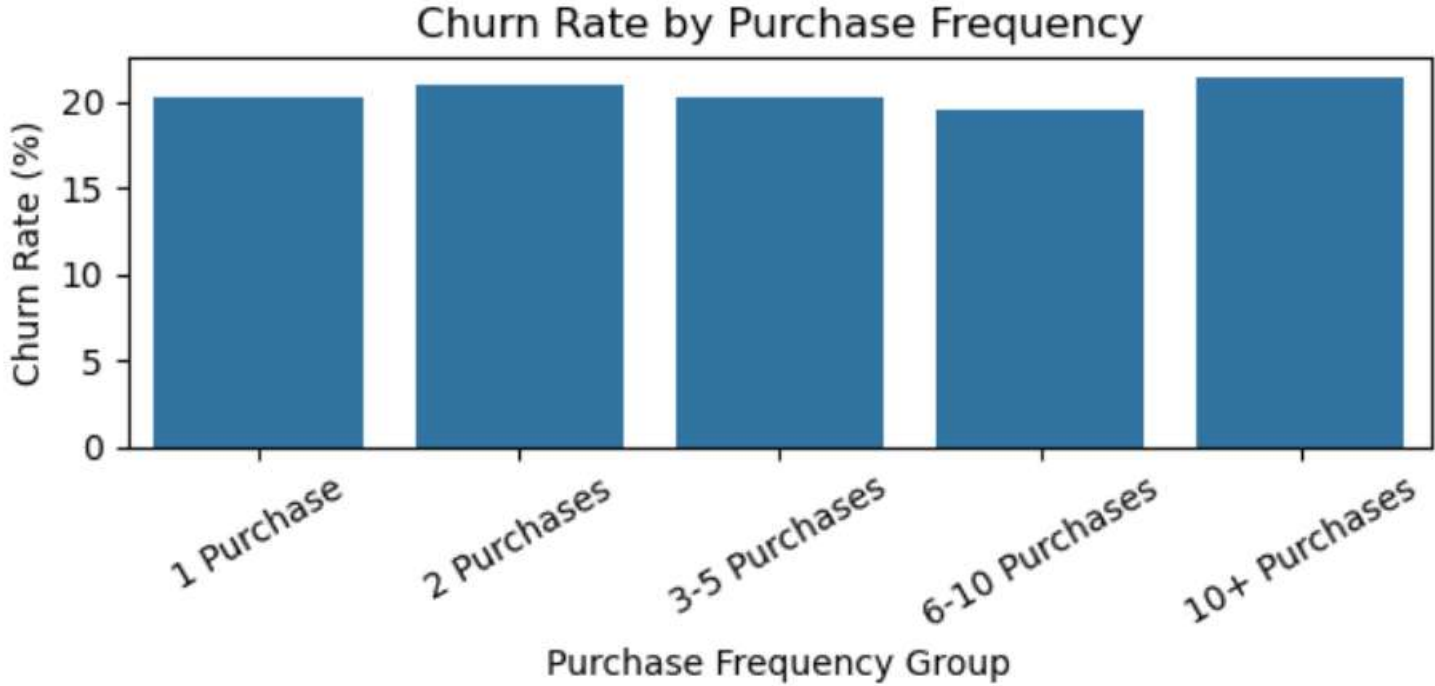
	Recency_Group	Customers	Churned_Customers	Churn_Rate
0	0-30 Days	5371	1086	20.22
1	31-60 Days	4717	969	20.54
2	61-90 Days	4129	824	19.96
3	91-180 Days	10249	1997	19.48
4	181-365 Days	12676	2566	20.24
5	365+ Days	12531	2500	19.95

Churn Rate by Recency Group



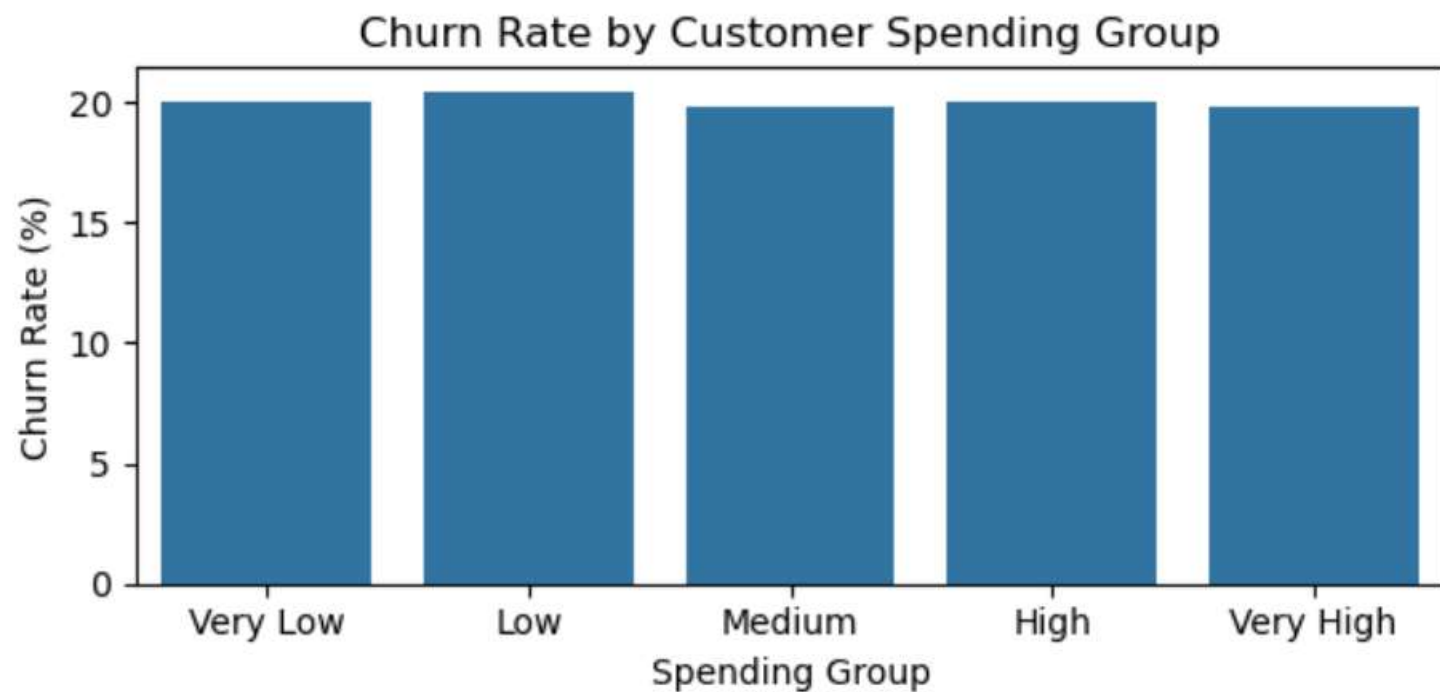
Churn by Frequency Group:

	Frequency_Group	Customers	Churned_Customers	Churn_Rate
0	1 Purchase	1685	340	20.18
1	2 Purchases	4277	896	20.95
2	3-5 Purchases	24525	4941	20.15
3	6-10 Purchases	18509	3620	19.56
4	10+ Purchases	677	145	21.42



Churn by Monetary Group:

	Monetary_Group	Customers	Churned_Customers	Churn_Rate
0	Very Low	9935	1988	20.01
1	Low	9934	2028	20.41
2	Medium	9935	1971	19.84
3	High	9934	1989	20.02
4	Very High	9935	1966	19.79



customer_age column is not available.

gender column is not available.

Churn by Recency Score:

	R_Score	Customers	Churned_Customers	Churn_Rate
0	1	9935	1968	19.81
1	2	9934	1963	19.76
2	3	9935	2036	20.49
3	4	9934	1946	19.59
4	5	9935	2029	20.42

Churn by Frequency Score:

	F_Score	Customers	Churned_Customers	Churn_Rate
0	1	9935	2014	20.27
1	2	9934	2015	20.28
2	3	9935	2003	20.16
3	4	9934	1926	19.39
4	5	9935	1984	19.97

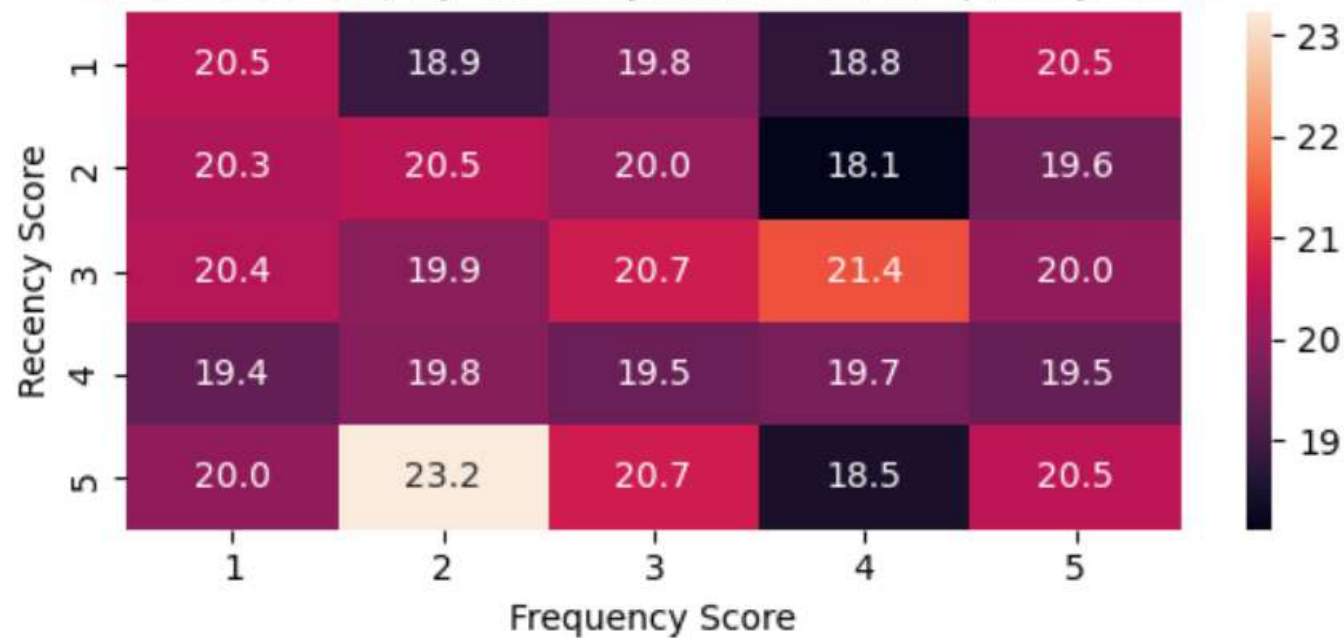
Churn by Monetary Score:

	M_Score	Customers	Churned_Customers	Churn_Rate
0	1	9935	1988	20.01
1	2	9934	2028	20.41
2	3	9935	1971	19.84
3	4	9934	1989	20.02
4	5	9935	1966	19.79

RFM Churn Heatmap Data:

F_Score	1	2	3	4	5
R_Score					
1	20.50	18.92	19.76	18.78	20.54
2	20.34	20.51	20.05	18.13	19.58
3	20.37	19.88	20.70	21.38	19.99
4	19.43	19.85	19.53	19.68	19.45
5	19.98	23.21	20.71	18.49	20.50

Churn Rate (%) by Recency Score and Frequency Score



High-Risk Customers: 27912

High-Risk Customers: 27912

	customer_id	recency	frequency	monetary	R_Score	F_Score	M_Score	RFM_Score	churn
46785	47087	104	17	45700	4	5	5	14	1
9346	9405	82	11	42479	4	5	5	14	1
6739	6786	14	12	42385	5	5	5	15	1
20720	20842	180	10	41501	3	5	5	13	1
20743	20865	34	11	41391	5	5	5	15	1
27976	28144	315	14	41347	2	5	5	12	0
34827	35037	349	11	41218	2	5	5	12	1
48150	48471	308	14	40436	2	5	5	12	0
20039	20158	236	12	40306	3	5	5	13	1
49008	49335	231	13	40210	3	5	5	13	1
33222	33424	150	11	40152	3	5	5	13	1
18913	19030	1	13	40034	5	5	5	15	1
6742	6789	294	11	39574	2	5	5	12	0
2233	2247	27	12	39314	5	5	5	15	1

35044	35256	291	11	39268	2	5	5	12	1
38089	38329	205	11	39101	3	5	5	13	1
31467	31659	368	10	39068	2	5	5	12	0
31269	31459	5	12	38847	5	5	5	15	1
26753	26912	132	11	38523	4	5	5	14	1
21345	21469	1	12	38518	5	5	5	15	1

High-Value At-Risk Customers: 4226

	customer_id	recency	frequency	monetary	R_Score	F_Score	M_Score	RFM_Score	churn
27976	28144	315	14	41347	2	5	5	12	0
48150	48471	308	14	40436	2	5	5	12	0
6742	6789	294	11	39574	2	5	5	12	0
31467	31659	368	10	39068	2	5	5	12	0
24833	24981	307	11	38511	2	5	5	12	0
28918	29096	430	13	37978	1	5	5	11	0
29108	29286	282	11	37958	2	5	5	12	0

28920	29098	245	11	37770	2	5	5	12	0
45686	45979	340	14	37593	2	5	5	12	0
26174	26328	319	13	37519	2	5	5	12	0
11186	11254	385	11	37233	2	5	5	12	0
40241	40504	446	11	37019	1	5	5	11	0
32023	32218	253	10	36837	2	5	5	12	0
46555	46855	255	10	36804	2	5	5	12	0
30830	31017	256	11	36309	2	5	5	12	0
34510	34720	534	11	36175	1	5	5	11	0
9716	9775	380	11	36166	2	5	5	12	0
43065	43344	311	12	36083	2	5	5	12	0
36735	36965	302	11	35844	2	5	5	12	0
41886	42161	268	13	35469	2	5	5	12	0

Churned High-Value Customers: 3955

	customer_id	recency	frequency	monetary	R_Score	F_Score	M_Score	RFM_Score
46785	47087	104	17	45700	4	5	5	14

9346	9405	82	11	42479	4	5	5	14
6739	6786	14	12	42385	5	5	5	15
20720	20842	180	10	41501	3	5	5	13
20743	20865	34	11	41391	5	5	5	15
34827	35037	349	11	41218	2	5	5	12
20039	20158	236	12	40306	3	5	5	13
49008	49335	231	13	40210	3	5	5	13
33222	33424	150	11	40152	3	5	5	13
18913	19030	1	13	40034	5	5	5	15
2233	2247	27	12	39314	5	5	5	15
35044	35256	291	11	39268	2	5	5	12
38089	38329	205	11	39101	3	5	5	13
31269	31459	5	12	38847	5	5	5	15
26753	26912	132	11	38523	4	5	5	14
21345	21469	1	12	38518	5	5	5	15
37918	38155	10	11	38405	5	5	5	15
23990	24135	28	13	38315	5	5	5	15
11130	11197	9	13	38162	5	5	5	15

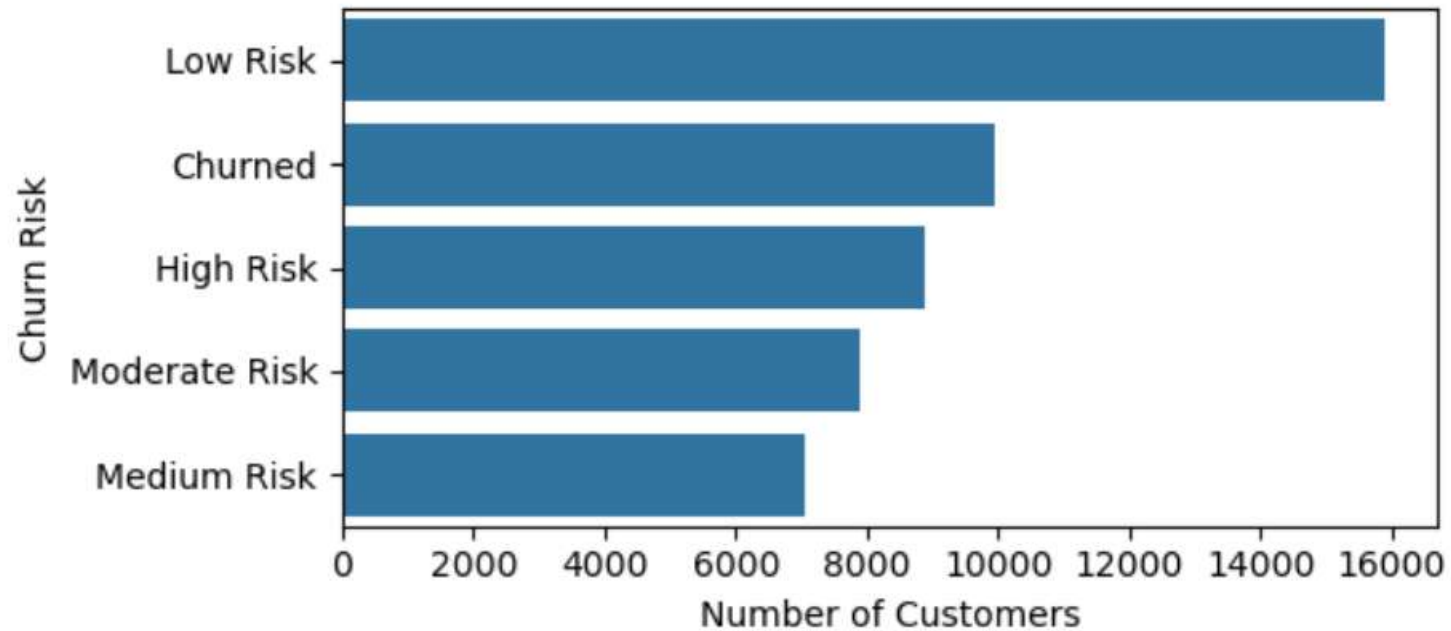
28410	28582	71	13	38123	4	5	5	14
-------	-------	----	----	-------	---	---	---	----

Churned Customer Revenue: 136036157
Active Customer Revenue: 545306526
Total Customer Revenue: 681342683
Churned Revenue Percentage: 19.97 %

Churn Risk Summary:

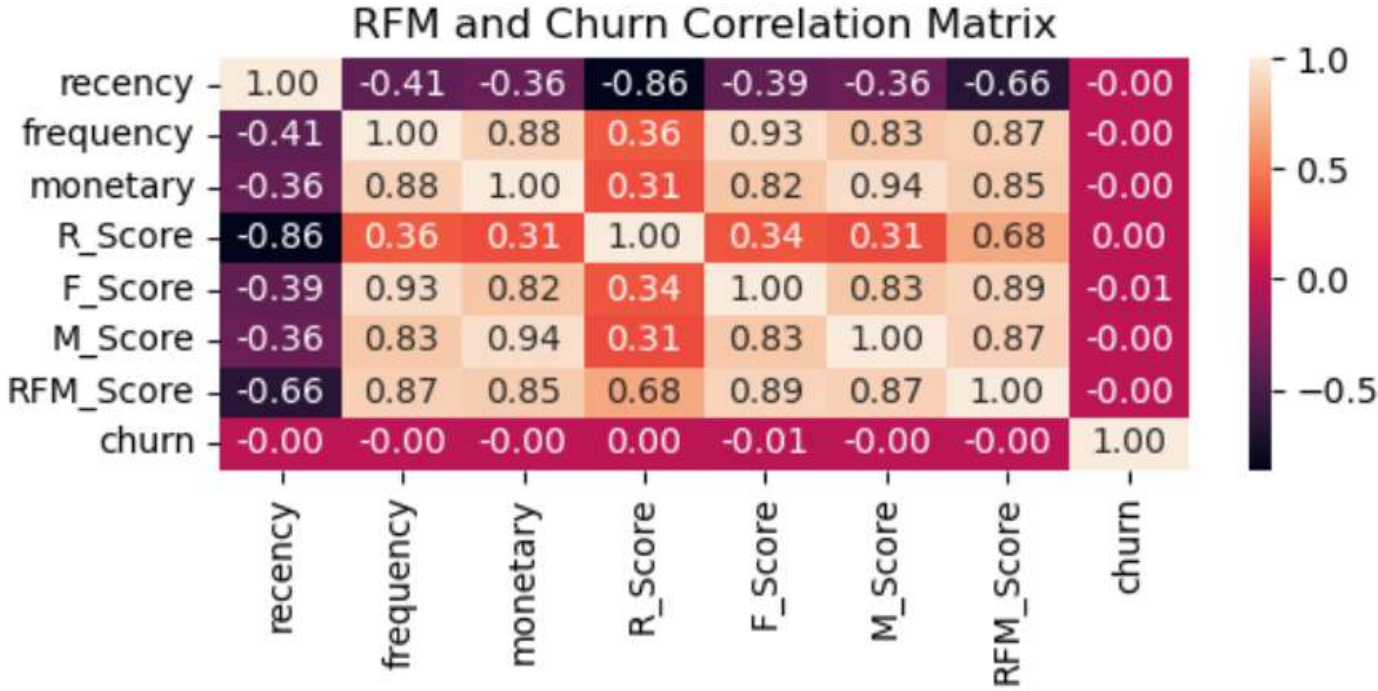
	Churn_Risk	Customer_Count	Percentage
0	Low Risk	15894	32.00
1	Churned	9942	20.01
2	High Risk	8887	17.89
3	Moderate Risk	7899	15.90
4	Medium Risk	7051	14.19

Customer Churn Risk Distribution



Correlation Matrix:

	recency	frequency	monetary	R_Score	F_Score	M_Score	RFM_Score	churn
recency	1.00	-0.41	-0.36	-0.86	-0.39	-0.36	-0.66	-0.00
frequency	-0.41	1.00	0.88	0.36	0.93	0.83	0.87	-0.00
monetary	-0.36	0.88	1.00	0.31	0.82	0.94	0.85	-0.00
R_Score	-0.86	0.36	0.31	1.00	0.34	0.31	0.68	0.00
F_Score	-0.39	0.93	0.82	0.34	1.00	0.83	0.89	-0.01
M_Score	-0.36	0.83	0.94	0.31	0.83	1.00	0.87	-0.00
RFM_Score	-0.66	0.87	0.85	0.68	0.89	0.87	1.00	-0.00
churn	-0.00	-0.00	-0.00	0.00	-0.01	-0.00	-0.00	1.00



Final Churn Analysis Summary:

	Metric	Value
0	Total Customers	4.967300e+04
1	Churned Customers	9.942000e+03
2	Non-Churned Customers	3.973100e+04
3	Overall Churn Rate (%)	2.001000e+01
4	Average Recency - Churned	2.585600e+02

5	Average Recency - Non-Churned	2.608900e+02
6	Average Frequency - Churned	5.020000e+00
7	Average Frequency - Non-Churned	5.040000e+00
8	Average Spending - Churned	1.368298e+04
9	Average Spending - Non-Churned	1.372496e+04
10	Churned Customer Revenue	1.360362e+08
11	Churned Revenue Percentage (%)	1.997000e+01
12	High-Risk Customers	2.791200e+04
13	High-Value At-Risk Customers	4.226000e+03
14	Churned High-Value Customers	3.955000e+03

Saved: customer_churn_analysis.csv

Saved: overall_churn_summary.csv

Saved: final_churn_summary.csv

Saved: RFM churn analysis files.

Saved: Recency, Frequency and Monetary group files.

Saved: RFM score component files.

Saved: rfm_churn_heatmap.csv

Saved: segment_churn_analysis.csv

Saved: High-risk customer files.

=====

STEP 5: CHURN ANALYSIS COMPLETED SUCCESSFULLY

=====

Total Customers: 49673

Churned Customers: 9942

Non-Churned Customers: 39731

Overall Churn Rate: 20.01 %

High-Risk Customers: 27912

High-Value At-Risk Customers: 4226

Churned High-Value Customers: 3955

Churned Customer Revenue: 136036157

=====

STEP 5 OUTPUT FILES

=====

1. customer_churn_analysis.csv
2. overall_churn_summary.csv
3. final_churn_summary.csv
4. rfm_churn_comparison.csv
5. rfm_score_churn_analysis.csv
6. recency_group_churn.csv
7. frequency_group_churn.csv
8. monetary_group_churn.csv
9. age_group_churn.csv
10. gender_churn_analysis.csv
11. recency_score_churn.csv
12. frequency_score_churn.csv
13. monetary_score_churn.csv
14. rfm_churn_heatmap.csv
15. segment_churn_analysis.csv
16. high_risk_customers.csv
17. high_value_at_risk_customers.csv
18. churned_high_value_customers.csv

Step 5 completed.